RETAINIFY — Employee Attrition Prediction System

Submitted by — Krithika M

Project Statement

The objective of this project is to analyze IBM HR workforce data to uncover employee attrition patterns, retention risk factors, and organizational trends using machine learning and explainable AI techniques. The goal is to help HR teams and business leaders identify at-risk employees early, understand the key drivers behind turnover, and take data-driven retention action through an interactive enterprise web platform.

Modules Implemented

1. Data Acquisition and Understanding
2. Data Cleaning and Preprocessing
3. Feature Engineering and Scaling
4. Model Training and Hyperparameter Tuning
5. Model Evaluation and Threshold Optimization
6. Attrition Driver Analysis
7. Employee Trends Visualization
8. Backend API Development
9. Frontend Dashboard
10. Deployment and Documentation

1. Data Acquisition and Understanding

The purpose of this module was to collect, load, and understand the raw dataset before analysis. This step ensured that the data was accurate, complete, and structured for efficient processing. By performing initial exploration and validation, the foundation was laid for preprocessing, feature engineering, and model training in later stages.

Dataset Overview

Dataset Name: raw-data.csv **Shape:** 1,470 rows × 35 columns

The raw dataset contains records of employee profiles across an organization, including demographic information, job role details, compensation data, satisfaction scores, performance ratings, and attrition status. It serves as the foundation for analyzing workforce patterns, attrition risk, and retention drivers.

Goals

- Build a clean and reliable data foundation for attrition prediction modeling.

- Define and extract key behavioral and organizational features to support machine learning.
- Ensure data quality by identifying missing values, constant columns, and structural inconsistencies.
- Store preprocessed data in a reusable format for faster downstream development.

Key Performance Indicators (KPIs)

KPI	Description
Attrition Rate	Proportion of employees who left over total workforce
Recall Score	Percentage of actual at-risk employees correctly identified
Risk Score	Predicted attrition probability scaled to 0–10
Replacement Cost	Estimated financial cost if high-risk employee departs
Workforce Health Index	Overall organizational retention score out of 100

Workflow

- Project scope established for attrition prediction and retention analysis.
- KPIs identified to measure model effectiveness and business impact.
- Workflow defined: Data ingestion → Preprocessing → Feature Engineering → Model Training → Evaluation → Deployment

---

Load CSV Using Pandas

Raw data was loaded from the IBM HR Analytics dataset using pandas.read_csv().

The dataset contains:

- **1,470 rows**
- **35 columns**

Exploring Schema, Types, Size, and Nulls

Check	Findings
Schema and Dtypes	Mix of int64 and object types; satisfaction and rating columns stored as integers
Size	~0.3 MB CSV file
Nulls	Zero missing values across all 35 columns confirmed
Duplicates	No duplicate employee records found
Constant Columns	EmployeeCount, StandardHours, Over18 contain single values throughout

---

Null Values Per Column

All 35 columns returned zero null values. The dataset was confirmed as complete with no imputation required at this stage.

Sampling and Memory Optimizations

- Dataset is small enough for full processing without sampling.
- Constant columns identified for removal to reduce feature dimensionality.
- Target variable Attrition confirmed as object type requiring binary encoding.

---

2. Data Cleaning and Preprocessing

This step cleaned the dataset by removing irrelevant columns, encoding the target variable, and confirming data integrity. The cleaned dataset served as the direct input for feature engineering and model training.

Remove Constant Columns

Four columns were dropped as they carry no analytical value:

- EmployeeCount — value of 1 for every record
- StandardHours — value of 80 for every record
- Over18 — value of Y for every record
- EmployeeNumber — unique identifier with no predictive relevance

---

Encode Target Variable

The Attrition column was mapped from string to binary:

- Yes → 1
- No → 0

Class distribution confirmed:

- Stayed (0): 1,233 employees — 83.9%
- Left (1): 237 employees — 16.1%

This confirmed significant class imbalance requiring SMOTE rebalancing before model training.

Confirm Cleaned Dataset

- Shape after cleaning: 1,470 rows × 31 columns
- Zero null values confirmed post-cleaning
- All column data types verified as appropriate for downstream processing
- Cleaned dataset saved as processed-data.csv

Feature Dictionary

Column	Description
Age	Employee age in years
Attrition	Target variable — 1 if left, 0 if stayed
BusinessTravel	Travel frequency — Non-Travel, Travel_Rarely, Travel_Frequently
DailyRate	Daily compensation rate
Department	Employee department — HR, R&D, Sales
DistanceFromHome	Commute distance in kilometres
Education	Education level — 1 to 5
EducationField	Field of education
EnvironmentSatisfaction	Work environment satisfaction — 1 to 4
Gender	Employee gender
HourlyRate	Hourly compensation rate
JobInvolvement	Job involvement score — 1 to 4
JobLevel	Seniority level — 1 to 5
JobRole	Employee job function
JobSatisfaction	Job satisfaction score — 1 to 4
MaritalStatus	Marital status — Single, Married, Divorced
MonthlyIncome	Monthly salary in USD
MonthlyRate	Monthly rate value
NumCompaniesWorked	Number of previous employers
OverTime	Whether employee works overtime — Yes or No
PercentSalaryHike	Annual salary increase percentage
PerformanceRating	Performance rating — 1 to 4
RelationshipSatisfaction	Relationship satisfaction — 1 to 4
StockOptionLevel	Stock option grant level — 0 to 3
TotalWorkingYears	Total years of professional experience
TrainingTimesLastYear	Number of trainings attended last year

Column	Description
WorkLifeBalance	Work-life balance score — 1 to 4
YearsAtCompany	Tenure at current organization
YearsInCurrentRole	Years in current job role
YearsSinceLastPromotion	Years since most recent promotion
YearsWithCurrManager	Years working under current manager

---

3. Feature Engineering and Scaling

This step created six new derived features to capture behavioral and career patterns not directly available in the raw data. Categorical variables were one-hot encoded and all features were standardized using StandardScaler to prepare the final model-ready dataset.

Engineered Features

Feature	Formula	Purpose
IncomePerYear	MonthlyIncome × 12	Annualized compensation for scale consistency
PromotionGap	YearsSinceLastPromotion ÷ YearsAtCompany	Career stagnation indicator
CompanyExperienceRatio	YearsAtCompany ÷ TotalWorkingYears	Organizational loyalty measure
ManagerStability	YearsWithCurrManager ÷ YearsAtCompany	Relationship consistency metric
WorkloadScore	(OverTime==Yes × 2) + (Travel_Frequently × 2) + (Travel_Rarely × 1)	Combined workload intensity index
SalaryGrowth	PercentSalaryHike ÷ YearsAtCompany	Compensation growth rate

---

One-Hot Encoding

Seven categorical columns were encoded using get_dummies with drop_first=True:

- BusinessTravel → BusinessTravel_Travel_Frequently, BusinessTravel_Travel_Rarely
- Department → Department_Research & Development, Department_Sales
- EducationField → five binary columns
- Gender → Gender_Male
- JobRole → eight binary columns
- MaritalStatus → MaritalStatus_Married, MaritalStatus_Single
- OverTime → OverTime_Yes

Feature Scaling

StandardScaler was fitted on the full feature matrix X and applied to all 50 columns. The scaler was saved as scaler.pkl for use during backend inference to ensure prediction consistency.

Final Dataset Shape

- Rows: 1,470
- Feature columns: 50
- Target column: Attrition
- Total shape: 1,470 × 51

Artifacts saved:

- feature_columns.pkl — ordered list of all 50 feature names
- scaler.pkl — fitted StandardScaler object
- feature-engineered-data.csv — scaled dataset for model training

---

4. Model Training and Hyperparameter Tuning

This step trained three classification models on the rebalanced dataset, applied GridSearchCV for hyperparameter optimization, and retrained final models on the full dataset for production deployment.

Class Imbalance Handling

SMOTE was applied to the training split to address the 84/16 class imbalance:

- Original training distribution: 986 stayed, 190 left
- Resampled training distribution: 986 stayed, 986 left

Models Trained

Model	Optimization Metric	Cross-Validation Folds
Logistic Regression	Recall	5-fold StratifiedKFold
Random Forest	Recall	5-fold StratifiedKFold
XGBoost	Recall	5-fold StratifiedKFold

Best Hyperparameters

Model	Best Parameters	Best CV Recall
Logistic Regression	C: 10, solver: lbfgs	82.1%

Model	Best Parameters	Best CV Recall
Random Forest	n_estimators: 100, max_depth: None, min_samples_split: 2	91.7%
XGBoost	learning_rate: 0.2, max_depth: 7, n_estimators: 100	90.97%

Production Retraining

Final models were retrained on the full 1,470-record dataset using the best parameters found during GridSearchCV, then saved as production artifacts:

- xgboost.pkl
- random-forest.pkl
- logistic-regression.pkl
- shap_explainer.pkl — TreeExplainer fitted on the final XGBoost model

---

5. Model Evaluation and Threshold Optimization

This step evaluated all three models using a custom classification threshold of 0.3 instead of the standard 0.5. The threshold was deliberately lowered because missing an employee who is about to leave (False Negative) is far more costly to the organization than raising a false alarm (False Positive). The system optimizes for Recall.

Custom Threshold Justification

At threshold 0.5, Recall scores ranged from 28% to 57% — unacceptable for HR use where every missed at-risk employee represents a real financial and operational loss. At threshold 0.3, Recall improves to 90%+ across all models, ensuring the platform catches the maximum number of flight risks.

Evaluation Results at Threshold 0.3

Model	Accuracy	Recall	Precision	F1 Score	AUC-ROC
Logistic Regression	57.5%	95.7%	30.2%	45.9%	81.3%
Random Forest	54.1%	97.9%	29.1%	44.8%	87.2%
XGBoost	53.1%	90.2%	28.4%	43.2%	89.7%

Confusion Matrix — XGBoost

	Predicted No	Predicted Yes
Actual No	130	117
Actual Yes	5	42

Model Selection Guidance

Model	When to Deploy
XGBoost	Balanced operational performance with optimal F1 score
Random Forest	Maximum risk coverage — minimum at-risk employees missed
Logistic Regression	Strict explainability — clear mathematical feature coefficients

---

6. Attrition Driver Analysis

This step analyzed the key features driving attrition predictions across all three models, computed attrition rates by workforce segment, and produced the critical signal data used in the Batch Analysis page of the platform.

Top Attrition Drivers — XGBoost Feature Importance

Rank	Feature	Importance
1	OverTime_Yes	16.0%
2	JobLevel	8.4%
3	StockOptionLevel	6.0%
4	TotalWorkingYears	5.5%
5	JobRole_Human Resources	4.0%
6	JobRole_Research Director	3.0%
7	YearsWithCurrManager	2.9%
8	JobSatisfaction	2.5%

Attrition Rate by Department

Department	Attrition Rate
Sales	Highest
Human Resources	Moderate
Research and Development	Lowest

Attrition Rate by Job Role

Job Role	Attrition Rate
Sales Representative	~40%

Job Role	Attrition Rate
Laboratory Technician	~24%
Human Resources	~23%
Research Scientist	~16%
Manager	~5%
Research Director	~2.5%

Attrition Rate by Overtime Status

Overtime	Attrition Rate
Yes	~31%
No	~10%

Employees working overtime are approximately 3× more likely to leave. Overtime is the single strongest controllable retention lever identified in the dataset.

Attrition Rate by Age Bracket

Age Bracket	Attrition Rate
18–24	Highest
25–34	Very high
35–44	Moderate
45–54	Low
55+	Lowest

Attrition Rate by Marital Status

Marital Status	Attrition Rate
Single	Highest
Divorced	Moderate
Married	Lowest

Critical System Signals

- Approximately 46.9% of high-risk employees are on mandatory overtime
- High-risk employees earn on average $1,400 less per month than low-risk employees

- Employees with Stock Option Level 0 show attrition rates more than 4× higher than Level 3
- The 25–35 age bracket represents the highest concentration of flight risk across all departments

---

Workforce Health Metrics

Metric	Value
Workforce Health Index	Score derived from high-risk employee percentage
Financial Liability	High-risk employee count × $50,000 average replacement cost
Attrition Rate	16.1% baseline across full dataset
Avg Replacement Cost per Employee	6× monthly salary

---

7. Employee Trends Visualization

This step produced the full suite of visual analyses used across the Dataset page and Batch Analysis page of the Retainify platform. All charts were generated from the IBM HR dataset using matplotlib and seaborn.

Visualizations Produced

Chart	Insight
Attrition distribution donut	83.9% stayed / 16.1% left baseline split
Attrition by overtime	31% attrition for overtime workers vs 10% for non-overtime
Attrition by department	Sales highest, R&D lowest
Attrition by job role	Sales Representative and Lab Technician most vulnerable
Age vs attrition histogram	25–35 bracket highest concentration of leavers
Monthly income by attrition	Leavers earn significantly less on average
Attrition by marital status	Single employees most likely to leave
Work-life balance vs attrition	Low scores strongly predict attrition
Job satisfaction vs attrition	Low satisfaction appears in majority of leaver profiles
Environment satisfaction vs attrition	Below-average satisfaction in leaver group
Years at company vs attrition	Shorter tenure strongly correlated with attrition
Stock option level vs attrition	Level 0 far more likely to leave than Level 2 or 3
Job level vs attrition	Entry-level employees show highest attrition rates
Feature correlation heatmap	OverTime_Yes highest positive correlation with attrition
XGBoost feature importance bar	Top 15 features ranked by model importance score

8. Backend API Development

The FastAPI backend serves as the intelligence layer of the platform, handling all prediction, simulation, and analytics requests from the React frontend.

Architecture

```
backend/
├── main.py
├── app/
│   ├── __init__.py
│   ├── routes/
│   │   ├── __init__.py
│   │   └── main.py
│   ├── services/
│   │   ├── __init__.py
│   │   └── prediction.py
│   └── ml/
│       ├── __init__.py
│       └── loader.py
├── trained_models/
│   ├── xgboost.pkl
│   ├── random-forest.pkl
│   ├── logistic-regression.pkl
│   ├── feature_columns.pkl
│   ├── scaler.pkl
│   └── shap_explainer.pkl
└── datasets/
    └── raw-data.csv
```

API Endpoints

Method	Endpoint	Purpose
POST	/upload	Upload and validate CSV or XLSX dataset
POST	/predict	Single employee attrition prediction

Method	Endpoint	Purpose
POST	/simulate	Real-time recalculation for retention simulator
POST	/batch-analysis	Batch prediction across all uploaded employees
GET	/performance	Return model evaluation metrics
GET	/feature-importance	Return XGBoost feature importance rankings
GET	/dataset-analytics	Return auto-computed workforce analytics

---

Preprocessing Pipeline

The backend preprocessor mirrors the exact feature engineering steps from training notebooks to ensure inference consistency:

- Reads raw form input values from the prediction request
- Computes all 6 engineered features using the same formulas
- Constructs one-hot encoded binary columns for all categorical variables
- Aligns columns to the saved feature_columns.pkl ordered list
- Applies scaler.transform() using the saved StandardScaler

SHAP Explanation

For every single prediction, the backend:

- Runs TreeExplainer.shap_values() on the scaled input
- Extracts per-feature SHAP impact values
- Sorts by absolute impact and returns the top 6 factors
- Labels each factor with a human-readable name and business description
- Tags each factor as Increases Risk or Lowers Risk based on SHAP sign

---

Recommendation Engine

Rule-based recommendations are generated from raw form input values:

Condition	Recommendation	Priority
OverTime == Yes	Reduce Overtime Exposure	High
MonthlyIncome < $4,000	Review Compensation Package	High
WorkLifeBalance <= 1	Improve Work-Life Balance	High
YearsSinceLastPromotion > 3	Accelerate Career Progression	High
JobSatisfaction <= 2	Enhance Job Engagement	Medium

Condition	Recommendation	Priority
EnvironmentSatisfaction <= 2	Improve Work Environment	Medium
StockOptionLevel == 0	Introduce Stock Incentives	Medium
BusinessTravel == Travel_Frequently	Optimize Travel Requirements	Medium

---

9. Frontend Dashboard

The React frontend provides an enterprise-grade interactive interface for HR teams to explore attrition predictions, workforce analytics, and model performance.

Application Structure

```
frontend/src/
├── App.tsx
├── index.tsx
├── context/
│   └── AppContext.tsx
├── services/
│   └── api.ts
├── components/
│   └── Navbar.tsx
└── pages/
    ├── HomePage.tsx
    ├── PredictPage.tsx
    ├── DatasetPage.tsx
    ├── PerformancePage.tsx
    └── BatchPage.tsx
```

Page Descriptions

Home Page Landing page explaining the Retainify platform, featuring an animated hero illustration with live risk gauges, four feature cards, and an About section covering the attrition problem statement.

Predict Page Primary prediction interface with a file upload overlay, intelligence model selector, personal profile form, job and career details form, and satisfaction sliders. After prediction the form is replaced by a two-column results layout — Risk Assessment and Prediction Explanation on the left, Retention Simulator and Recommendations on the right. The Retention Simulator calls /simulate on every slider change for real-time probability updates.

Dataset Page Automatic workforce analytics page loaded from /dataset-analytics. Displays KPI bar, attrition overview, 10 charts covering distribution and breakdown across all key features, a full EDA section with imbalance analysis, age vs attrition chart, role vulnerability index, overtime over-index chart, and a dataset profile summary table.

Model Performance Page Technical validation page displaying the threshold optimization strategy with a line chart, business justification for the 0.3 threshold, recall comparison bar chart, ROC/AUC multi-model curve, accuracy vs recall trade-off chart, statistical evaluation table, and a model selection guidance matrix.

Batch Analysis Page Workforce command center loaded automatically from /batch-analysis. Displays four KPI cards, risk segment breakdown, workforce health index, financial liability estimate, departmental risk heatmap, job role vulnerability ranking, critical system signals, suggested action plan, a searchable and sortable employee results table with pagination, and workforce retention recommendations.

State Management

Global state managed through React Context API preserves:

- datasetUploaded — controls upload overlay visibility and page unlock
- predictionResult — shared prediction data across Predict page components
- batchResults — batch analysis results for Batch page
- selectedModel — active model selection persisted across interactions
- datasetInfo — uploaded file metadata

---

10. Deployment and Documentation

Deployment Architecture

Layer	Platform	Details
Frontend	Vercel	React production build, CDN-served, automatic GitHub deploys
Backend	Render	FastAPI Web Service, uvicorn server, 750 hours free tier
Version Control	GitHub	Full source code, pkl files, and dataset committed to main branch

---

Environment Configuration

Frontend environment variable set in Vercel:

- REACT_APP_API_URL — points to the Render backend URL

Backend startup sequence on Render:

- Build command: pip install -r requirements.txt
- Start command: uvicorn main:app --host 0.0.0.0 --port $PORT
- On startup: load_all() loads all six pkl files from trained_models/
- On startup: raw-data.csv auto-loaded from datasets/ for instant analytics

---

Deliverables

- Live production frontend — Vercel URL
- Live production backend — Render URL
- GitHub repository with full source code, notebooks, and trained models
- Six production pkl artifacts in trained_models/
- IBM HR raw dataset in datasets/
- Seven Jupyter notebooks covering the full ML pipeline
- README with setup instructions, architecture overview, and deployment guide
- UML class diagram, component diagram, and project workflow diagram
- Implementation plan document
